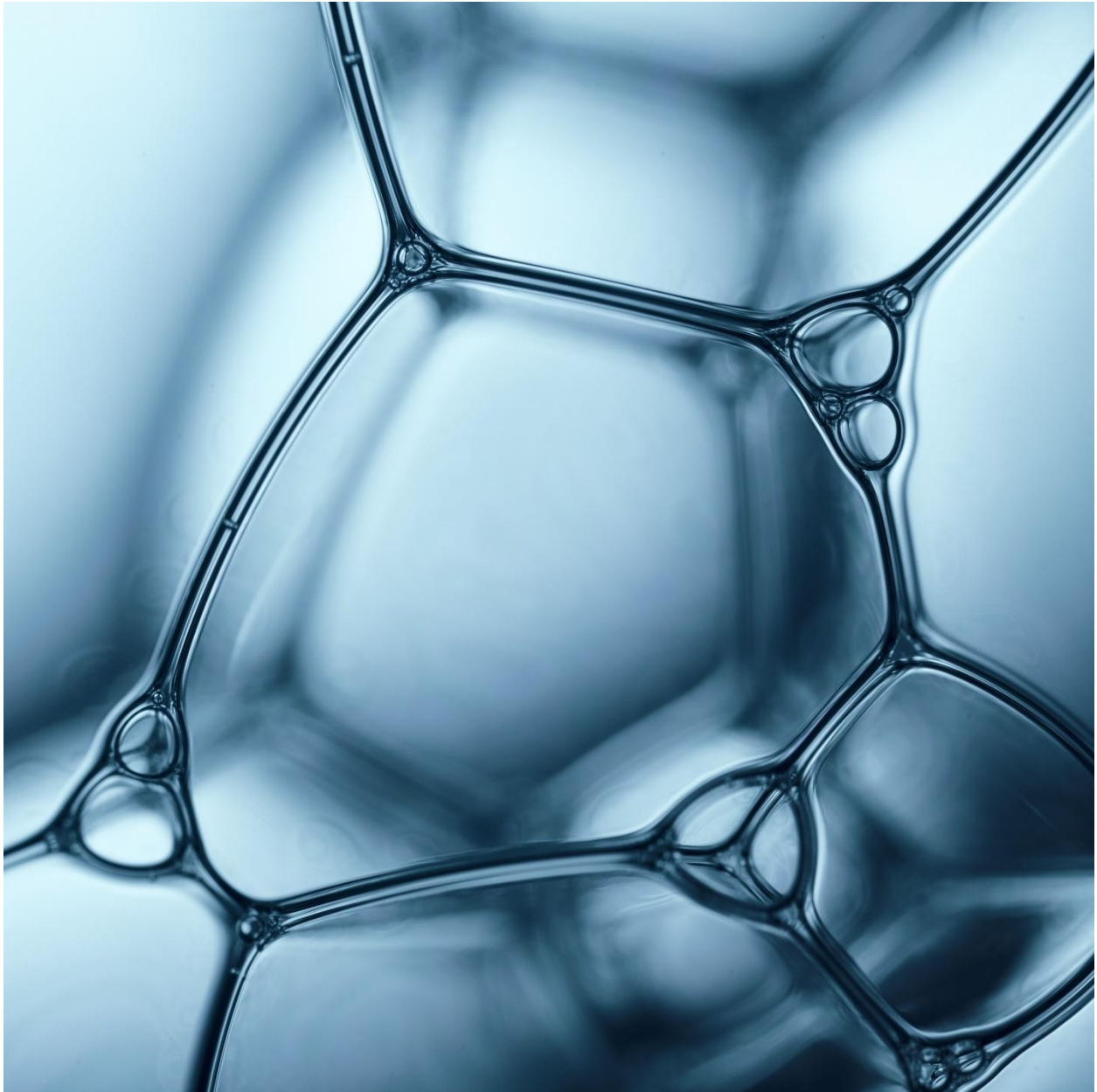

S&P Global

Market Intelligence

CIQ Python SDK User Guide



Client Support

Email: support.api.mi@spglobal.com <i>File delivery support is available 24/7/365</i>			
U.S. / Canada / South America	Sales & Support: 1.800.523.4534 or 1.303.721.4802	China/ Hong Kong	+86.10.6569.2985
Mexico	303.721.4802	Japan	+81.3.4550.8350
Europe	+44. 20.7176.8580	South Korea	+82.2.2022.2395

Contents

Overview.....	1
Authentication	1
Overview	1
Features	1
Financials	2
Overview	2
Features	2
Market Data.....	2
Overview	2
Features	2
Data Widgets Library.....	2
Overview	2
Features	2
Efficiency and Cross-Platform Compatibility.....	2
Current API Python Client Library Versions.....	3
Minimum Supported Frameworks and Language Versions.....	3
Setting Up the SDK.....	4
Setting Up and Running SDK in PyCharm.....	4
Install PyCharm	4
Create a New Project.....	4
Install SDK Package.....	4
Create a Python Script.....	4
Running Python Script	4
Setting-Up Jupyter Notebook.....	5
Download Anaconda.....	5
Install Anaconda	5
Setting Up Jupyter Notebook	5
Installing the Python SDK.....	6
Installing Python SDK in a Jupyter Notebook.....	6
Installing Python SDK in a DataBricks Notebook	6

Overview of Functions	8
Request Parameters.....	8
Authentication Functions.....	9
Overview	9
Functions.....	9
Financial Services Functions.....	10
Overview	10
Functions.....	10
Market Data Functions	12
Overview	12
Functions.....	12
Data Widgets Functions.....	13
Overview	13
Functions.....	14
Company Name Match Functions.....	17
Overview	17
Functions.....	17
Using the SDK & Code Samples	19
Set Up	19
Fetching Data	20
Financials Code Samples.....	20
Market Data Code Samples.....	21
CIQ Data Widget & Company Name Match Code Samples.....	22
Complete Example Code.....	23
Verifying SDK File Integrity.....	26
Compute the SHA-256 Hash.....	26
Verify the Hash.....	26
Best Practices	27
Use of Environment Variables and .env Files	27
Use of .env Files.....	27
Load .env Variables	27

Preferred Identifier Types	28
Throttling Limits	28
Additional Notes	28
Revision History	29

Overview

The CIQ API Python Library provides a simplified, low code solution to using the S&P Capital IQ API, elevating the user experience through offering simplified functions to drive the process of making a request.

The library includes comprehensive functions for seamless authentication, retrieval of detailed point-in-time and historical financial data, and access to end-of-day and time-series market data essential for analysis and decision-making.

This guide provides all the information required to use the CIQ API Python Library. We will explore the current Python library versions, minimum supported frameworks, and detailed, step-by-step instructions on how to authenticate a user and retrieve both financial and market data. Best practices are also outlined to ensure optimal use of the product.

We have outlined the key features of each group of functionalities below on a high level.

Authentication

Overview

This library provides functionalities enabling a seamless authentication process to securely access API services. Included are methods for obtaining and refreshing access tokens while ensuring all API requests are properly authorized.

Features

- Secure authentication with methods for obtaining access tokens.
- Ability to maintain continuous API access with token refresh functionality.

Financials

Overview

This library provides a number of functionalities enabling you to retrieve point-in-time and historical financial data including income statements, balance sheets, cash flow statements, and other financial metrics essential for comprehensive financial analysis.

Features

- Retrieve point-in-time and historical financial statements (income statements, balance sheets, cash flow statements).
- Access specific financial metrics for detailed analysis.

Market Data

Overview

This library provides several functionalities enabling you to access end-of-day and time-series market data, including stock prices, trading volumes, dividend information, and other market-related information crucial for analysis and decision-making.

Features

- Fetch end-of-day and time-series market data for specified identifiers.
- Retrieve historical market data over specified periods for analysis.

Data Widgets Library

Overview

This Python library offers a range of functions used to retrieve financial highlights, market data, corporate data, ESG scores and more. The data retrieved from these functions reflects information obtained from the CIQ Pro Platform data widgets.

Note that most datapoints are reflected, but some are not due to contractual obligations and other various reasons; due to this, there may be slight differences in the data points offered on the platform versus through this library. For more information, contact client support.

Features

- Fetch financial highlights, market data, corporate data, ESG scores, and other key data points.
- Easily integrate S&P Capital IQ Pro company profile data into your workflow.

Efficiency and Cross-Platform Compatibility

This SDK is designed to work efficiently within Python environments, leveraging the language's strengths in data processing and scripting. Python's extensive libraries and community support make it an ideal choice for developing robust and scalable applications.

Current API Python Client Library Versions

Library	Version	Download
spgmi_api_sdk_python-3.0.0.tar.gz	3.0.0	Download Link
spgmi_api_sdk_python-2.0.0.tar.gz	2.0.0	Download Link

Minimum Supported Frameworks and Language Versions

Software	Minimum Supported Version
Python	>=3.9.0,<4.0.0

Setting Up the SDK

Setting Up and Running SDK in PyCharm

Install PyCharm

- Visit the JetBrains website download page - [PyCharm-Download](#).
- Scroll Down after hitting Download page to find the Community Edition of PyCharm for your operating system and download.

Create a New Project

- Open PyCharm and select "New Project."
- Choose a location for your project.
- Ensure you have the appropriate Python version installed that is compatible with the SDK.
- After setup, configure Python Interpreter from the Settings window, navigate to Project: <Your_Project_Name> Python Interpreter.

Install SDK Package

- Open the terminal within PyCharm (View > Tool Windows > Terminal).
- Use pip to install the required SDK package. For example:

```
pip install "--path--/spgmi_api_sdk_python-2.0.0.tar.gz"
```

Create a Python Script

- Right-click the project folder and select New > Python File.
- Name your file (e.g., __init__.py).

Running Python Script

- Right click on the Python file and click "run" from the dropdown menu.

Setting-Up Jupyter Notebook

Download Anaconda

- Visit the Anaconda Website:
 - Open your web browser and go to the Anaconda distribution page - Anaconda Download.
- Select Your Operating System:
 - Choose the installer for your operating system (Windows, macOS, or Linux).
- Download the Installer:
 - Click the download button for the Python 3.x version (the latest version is recommended).

Install Anaconda

- Run the Installer:
 - Locate the downloaded .exe file and double-click to start the installation process.
- Follow the Installation Steps:
 - Click "Next" to continue through the setup steps.
 - Agree to the license agreement.
 - Choose the installation type: "Just Me" or "All Users".
 - Choose the installation location).
- Advanced Installation Options:
 - Check the box to add Anaconda to your PATH environment variable (recommended).
 - Check the box to register Anaconda as your default Python Editor (recommended).
 - Click Install.
- Finish the Installation:
 - Once the installation is complete, click "Next", then "Finish".

Setting Up Jupyter Notebook

- Open Anaconda Navigator:
 - Windows: Open the Start Menu and search for "Anaconda Navigator".
 - macOS: Use Spotlight to search for "Anaconda Navigator".
 - Linux: Open a terminal and type anaconda-navigator.
- Launch Jupyter Notebook:
 - In Anaconda Navigator, find the Jupyter Notebook option and click the "Launch" button.
- Using Jupyter Notebook:
 - A new browser tab should open with the Jupyter Notebook interface.
 - You can create a new notebook by clicking the "New" button and selecting "Python 3".

Installing the Python SDK

Installing Python SDK in a Jupyter Notebook

To start using our Python SDK in a Jupyter Notebook, please follow the below steps:

1. Create a New Python 3 Notebook:

- Open Jupyter Notebook.
- Create a new Python 3 notebook file (. ipynb).

2. Install the Python SDK:

- Ensure you have the correct path to the SDK tarball file (. tar.gz). When you download the zip file, **please extract the tar.gz file to install the Python SDK**. The checksum.sha256 file is for integrity checks.
- Use the following commands to install all the SDKs. Replace <path_to_sdk> with the actual path to the tarball file.

```
pip install "--path--/spgmi_api_sdk_python-3.0.0.tar.gz"
```

3. Restart the Kernel:

- After installing each SDK, it is important to restart the Jupyter Notebook kernel to ensure the newly installed library is properly loaded.
- You can restart the kernel by going to the menu bar and selecting Kernel > Restart.

Installing Python SDK in a DataBricks Notebook

To start using our Python SDK in a DataBricks Notebook, please follow the below steps:

Step 1: Upload SDK Libraries to Databricks Workspace:

- Navigate to Workspace:
 - In the Databricks workspace dashboard, click on the "Workspace" button on the left-hand side.
- Create Folder (Optional):
 - Right-click on the workspace directory where you want to upload the files and select "Create" -> "Folder" to create a new folder.
- Upload Files:
 - Click on the desired folder to upload files to. Click the "Upload" button and select the SDK tarball files (. tar.gz) from your local machine.

Step 2: Installing Python SDK in Databricks Using pip:

- Create or Open Notebook:
 - Create or open an existing notebook in the Databricks workspace.
- Install SDK Library:

- Ensure you have the correct path to the SDK tarball file (.tar.gz). When you download the zip file, **please extract the tar.gz file to install the Python SDK.** The checksum.sha256 file is for integrity checks.
- In the first cell of your notebook, use the following commands to install the SDK from the workspace. Replace <path_to_sdk> with the actual path where the SDK files were uploaded.

```
# Example commands to install SDKs using pip and the  
workspace path  
pip install "--path--/spgmi_api_sdk_python-3.0.0.tar.gz"
```

Overview of Functions

This Python SDK provides functions to authenticate and easily retrieve financial and market data. Each function has mandatory and optional parameters.

Request Parameters

The main parameters of a typical request include:

Request Parameters	Description
username and password	Your login credentials (username and a link for password set up) will be sent in a welcome email. Your username and password are confidential login credentials, please treat them as such. To reset your password, reach out to our Enterprise API Support team at support.api.mi@spglobal.com and they will send a password reset via email.
refresh_token	This token is used to refresh the authentication obtained from <code>get_token</code> . Within the <code>get_refresh_token()</code> function, pass the token retrieved from the <code>get_token</code> function to create a new access token.
token	Token used for authentication obtained from <code>get_token()</code> .
identifiers	This library supports company and security-level identifiers. Supported identifiers are CUSIP6_CINS6, CUSIP9_CINS9, ISIN, TICKER, GVKEY, SEDOL, CIQ_ID, ticker:exchange, CIQTradingItemID. For datasets like Agency

	Ratings, ESG Scores, and Latest Activity the tickers are not supported. A combination of identifier types can be used in comma delimited format.
properties	Each function has an independent group of supported properties. You will find the list of properties below each function. Refer to the Properties Matrix document for more information on specific properties. Note that Data Widget functions do not support properties.
proxy	The proxy parameter is an optional argument that allows users to specify a proxy server for the request. This can be useful in environments where internet access is restricted or monitored, such as corporate networks. If not specified, the function will not use a proxy.

Below is a list of functions associated with the Python SDK and their supported properties (where applicable.) The (*) denotes mandatory fields.

Authentication Functions

Overview

The Authentication Service includes functions to authenticate users and generate tokens necessary for accessing other data services.

Functions

1. `get_token ()`:
 - **Purpose:** Authenticates the user with the provided username and password and returns a JSON response containing the access token.
 - **Usage:** Use this function to obtain an access token which will be required for all subsequent API calls. For getting only access_token use `"get_token().get('access_token')"`
2. `get_refresh_token(refresh_token*)`:
 - **Purpose:** Authenticates the user by refreshing the provided token obtained from the `get_token ()` function. This function returns a JSON response containing the new access token
 - **Usage:** Use this function to obtain an access token which will be required for all subsequent API calls. For getting only access_token use `"get_refresh_token(refresh_token*).get('access_token')"`

Financial Services Functions

Overview

This Python library provides several functions to retrieve point-in-time and historical financial services data including income statements, balance sheets, and cash flow statements. Functions in this library will return data in the form of pandas DataFrames.

Functions

- `get_income_statement_pit(identifiers*: list, properties: dict = {}, token: str) -> pd.DataFrame:`
 - Purpose:** Returns a list of income statement data values based on a single period.
 - Usage:** Use this function to get point-in-time income statement data.
 - Properties:** `periodType`, `asOfDate`, `filingMode`, `restatementTypeld`, `currencyId`, `currencyConversionModelId`
 - Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: `IQ_FILINGDATE_IS`, `IQ_EBITDA`, `IQ_NI`, `IQ_TOTAL_REV`, `IQ_EBIT`, `IQ_COGS`, `IQ_GROSS_MARGIN`, `IQ_DILUT_EPS_EXCL`, `IQ_GP`, `IQ_PERIODDATE`.
- `get_income_statement_historical(identifiers*: list, properties: dict = {}, token: str) -> pd.DataFrame:`
 - Purpose:** Returns an array of income statement data values based on a range of periods.
 - Usage:** Use this function to get historical income statement data over a specified period.
 - Properties:** `periodType`, `asOfDate`, `filingMode`, `restatementTypeld`, `currencyId`, `currencyConversionModelId`
 - Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: `IQ_FILINGDATE_IS`, `IQ_EBITDA`, `IQ_NI`, `IQ_TOTAL_REV`, `IQ_EBIT`, `IQ_COGS`, `IQ_GROSS_MARGIN`, `IQ_DILUT_EPS_EXCL`, `IQ_GP`, `IQ_PERIODDATE`.
- `get_balance_sheet_pit(identifiers*: list, properties: dict = {}, token: str) -> pd.DataFrame:`
 - Purpose:** Returns a list of balance sheet data values based on a single period.
 - Usage:** Use this function to get point-in-time balance sheet data.
 - Properties:** `periodType`, `asOfDate`, `filingMode`, `restatementTypeld`, `currencyId`, `currencyConversionModelId`
 - Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: `IQ_TOTAL_ASSETS`, `IQ_TOTAL_DEBT`, `IQ_NET_DEBT`, `IQ_CASH_ST_INVEST`, `IQ_TOTAL_COMMON_EQUITY`, `IQ_CASH_EQUIV`, `IQ_TOTAL_LIAB`, `IQ_LT_DEBT`, `IQ_PERIODDATE`.

4. `get_balance_sheet_historical(identifiers*: list, properties: dict = {}, token: str) -> pd.DataFrame:`

- **Purpose:** Returns an array of balance sheet data values based on a range of periods
- **Usage:** Use this function to get historical balance sheet data over a specified period.
- **Properties:** `periodType`, `asOfDate`, `filingMode`, `restatementType`, `currencyId`, `currencyConversionModel`
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: `IQ_TOTAL_ASSETS`, `IQ_TOTAL_DEBT`, `IQ_NET_DEBT`, `IQ_CASH_ST_INVEST`, `IQ_TOTAL_COMMON_EQUITY`, `IQ_CASH_EQUIV`, `IQ_TOTAL_LIAB`, `IQ_LT_DEBT`, `IQ_PERIODDATE`.

5. `get_cash_flow_pit(identifiers*: list, properties: dict = {}, token: str) -> pd.DataFrame:`

- **Purpose:** Returns a list of cash flow statement data values based on a single period.
- **Usage:** Use this function to get point-in-time cash flow data.
- **Properties:** `periodType`, `asOfDate`, `filingMode`, `restatementType`, `currencyId`, `currencyConversionModel`
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: `IQ_FILINGDATE_CF`, `IQ_CASH_OPER`, `IQ_CAPEX`, `IQ_PERIODDATE`.

6. `get_cash_flow_historical(identifiers*: list, properties: dict = {}, token: str) -> pd.DataFrame:`

- **Purpose:** Returns an array of cash flow statement data values based on a range of periods.
- **Usage:** Use this function to get historical cash flow data over a specified period.
- **Properties:** `periodType`, `asOfDate`, `filingMode`, `restatementType`, `currencyId`, `currencyConversionModel`
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: `IQ_FILINGDATE_CF`, `IQ_CASH_OPER`, `IQ_CAPEX`, `IQ_PERIODDATE`.

7. `get_financials_pit(identifiers*: list, mnemonics*: list, properties: dict = {}, token: str) -> pd.DataFrame:`

- **Purpose:** Returns point-in-time financials data based on requested mnemonics for a single period.
- **Usage:** Use this function to get point-in-time financial data for a variety of data points based on your specific use case.
- **Properties:** `periodType`, `asOfDate`, `filingMode`, `restatementType`, `currencyId`, `currencyConversionModel`
- **Mnemonics:** This function enables you to input a variety of mnemonics based on your use case. Refer to the [API Mnemonics List](#) for more information on supported mnemonics.

8. `get_financials_historical(identifiers*: list, mnemonics*: list, properties: dict = {}, token: str) -> pd.DataFrame:`
 - **Purpose:** Returns historical financial data based on requested mnemonics for a range of periods.
 - **Usage:** Use this function to get historical financial data for a variety of data points based on your specific use case.
 - **Properties:** `periodType`, `asOfDate`, `filingMode`, `restatementTypeId`, `currencyId`, `currencyConversionModelId`
 - **Mnemonics:** This function enables you to input a variety of mnemonics based on your use case. Refer to the [API Mnemonics List](#) for more information on supported mnemonics.

Market Data Functions

Overview

This Python library offers a range of functions to fetch market data, including pricing information, dividend data, and market metrics. Functions in this library will return data in the form of pandas DataFrames.

Functions

1. `get_pricing_info_pit(identifiers*: list, properties: dict = {}, token: str) -> pd.DataFrame:`
 - **Purpose:** Returns a list of pricing values for a single date.
 - **Usage:** Use this function to get current pricing data for given financial instruments.
 - **Properties:** `asOfDate`, `currencyId` `currencyConversionModelId`
 - **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: `IQ_CLOSEPRICE`, `IQ_CLOSEPRICE_ADJ`, `IQ_VOLUME`, `IQ_OPENPRICE`, `IQ_HIGHPRICE`, `IQ_LOWPRICE`.
2. `get_pricing_info_time_series(identifiers*: list, properties: dict = {}, token: str) -> pd.DataFrame:`
 - **Purpose:** Returns an array of pricing values for a range of dates based on a frequency.
 - **Usage:** Use this function to get historical pricing data over a specified period.
 - **Properties:** `startDate`, `endDate`, `frequency`, `currencyId`, `currencyConversionModelId`
 - **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: `IQ_CLOSEPRICE`, `IQ_CLOSEPRICE_ADJ`, `IQ_VOLUME`, `IQ_OPENPRICE`, `IQ_HIGHPRICE`, `IQ_LOWPRICE`.
3. `get_dividend_info_pit(identifiers*: list, properties: dict = {}, token: str) -> pd.DataFrame:`
 - **Purpose:** Returns a list of dividends for a single date.
 - **Usage:** Use this function to get current dividend information.
 - **Properties:** `currencyId` `currencyConversionModelId`

- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: IQ_DIV_SHARE, IQ_LAST_SPLIT_DATE, IQ_LAST_SPLIT_FACTOR.
4. `get_dividend_info_time_series(identifiers*: list, properties: dict = {}, token: str) ->pd.DataFrame:`
 - **Purpose:** Returns an array of dividends based on a range of dates.
 - **Usage:** Use this function to get historical dividend data over a specified period.
 - **Properties:** startDate, endDate, currencyId, currencyConversionModelId
 - **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: IQ_XDIV_DATE_LIST, IQ_DIV_RECORD_DATE_LIST, IQ_DIV_PAYMENT_DATE_LIST, IQ_DIV_AMOUNT_LIST, IQ_DIV_PAYMENT_TYPE_LIST.
 5. `get_market_info_pit(identifiers*: list, properties: dict = {}, token: str) ->pd.DataFrame:`
 - **Purpose:** Returns a list of market data values for a single date.
 - **Usage:** Use this function to get current market metrics.
 - **Properties:** asOfDate, currencyId currencyConversionModelId
 - **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: IQ_DIVIDEND_YIELD, IQ_MARKETCAP, IQ_TEV, IQ_SHARES_OUTSTANDING.
 6. `get_market_info_time_series(identifiers*: list, properties: dict = {}, token: str) -> pd.DataFrame:`
 - **Purpose:** Returns an array of market data values for a given date range.
 - **Usage:** Use this function to get historical market data over a specified period.
 - **Properties:** startDate, endDate, frequency, currencyId, currencyConversionModel
 - **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: IQ_DIVIDEND_YIELD, IQ_MARKETCAP, IQ_TEV, IQ_SHARES_OUTSTANDING.

Data Widgets Functions

Overview

This Python library offers functions that obtain information found on data widgets visible on the CIQ Pro Platform. Provided functions retrieve financial highlights, market data, corporate data, ESG scores and more. Data Widget functions do not support properties. Functions in this library will return data in the form of pandas DataFrames.

Note that while most datapoints are reflected, some are not due to contractual obligations and other various reasons; due to this, there may be slight differences in the data points offered on the platform versus through this library. For more details, contact client support.

Functions

1. `get_financials (identifiers*: list, period_type: str, token: str) -> pd.DataFrame:`

- **Purpose:** Returns an array of financial highlights data values for a defined period type including income statements, balance sheets, and cash flow statements.
- **Usage:** Use this function to get financial highlights data for a defined period type. Inputs for the `period_type` parameter can be found in the [Properties Matrix](#).
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: `IQ_PERIODDATE`, `IQ_TOTAL_REV`, `IQ_EBITDA`, `IQ_EBIT`, `IQ_NI`, `IQ_CAPEX`, `IQ_TOTAL_ASSETS`, `IQ_TOTAL_LIAB`, `IQ_TOTAL_EQUITY`, `IQ_CASH_ST_INVEST`, `IQ_NET_DEBT`, `IQ_BV_SHARE`, `IQ_RETURN_ASSETS`, `IQ_RETURN_EQUITY`.

2. `get_market_data (identifiers*: list, token: str) -> pd.DataFrame:`

- **Purpose:** Returns an array of end-of-day market data values including dividend, closing price, VWAP, open price, day high/low, and more.
- **Usage:** Use this function to get market data over a specified period.
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: `IQ_CLOSEPRICE`, `IQ_VWAP`, `IQ_OPENPRICE`, `IQ_HIGHPRICE`, `IQ_LOWPRICE`, `IQ_YEARHIGH`, `IQ_YEARLOW`, `IQ_BETA_5YR`, `IQ_MARKETCAP`, `IQ_TEV`, `IQ_VOLUME`, `IQ_SHARESOUTSTANDING`, `IQ_FLOAT_PERCENT`, `IQ_DIVIDEND_YIELD`.

3. `get_corporate_data(identifiers*: list, token: str) -> pd.DataFrame:`

- **Purpose:** Returns entity-level details including company description, contact information, key officers and directors, industry classification and other key details.
- **Usage:** Use this function to get information about an entity.
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: `IQ_COMPANY_STATUS`, `IQ_PRIMARY_INDUSTRY`, `BE CRS_ENTITY_BASE`, `IQ_YEAR_FOUNDED`, `IQ_PRIMARY_SIC_CODE`, `BE CRS_ENTITY_BIC_CODE`, `IS CRS_NAICS`, `IQ_BUSINESS_DESCRIPTION`, `IQ_COMPANY_ADDRESS`, `IQ_COMPANY_PHONE`, `IQ_COMPANY_WEBSITE`, `IQ_PROFESSIONAL-IQ_PROFESSIONAL_TITLE`, `IQ_GICS_CODE`, `IQ_INDUSTRY_SECTOR`, `IQ_INDUSTRY_GROUP`, `IQ_INDUSTRY`, `IQ_PRIMARY_INDUSTRY`, `IQ_TR_OFFER_DATE`.

4. `get_esg_scores(identifiers*: list, assessment_year: str, token: str) -> pd.DataFrame:`

- **Purpose:** Returns granular sustainability-related data points related to ESG Analytics, Climates and Environment Analytics.
- **Usage:** Use this function to obtain information about a deep range of environmental, social and governance data comparable across peers and subindustries.

- This function does not retrieve data points, but instead retrieves pre-defined data points. More information on these data points can be found on the XpressAPI Swagger page for [ESG Scores](#).
 - The following data points are retrieved for ESG Analytics: esgScore, envScore, socialScore, geScore, criteriaName (Climate Strategy, Emissions, Environmental Policy & Management Systems, Resource Efficiency and Circularity, Waste, Water, Customer Relationship Management, Human Capital Development, Human Rights, Labor Practice Indicators, Occupational Health & Safety, Privacy Protection, Talent Attraction & Retention, Governance & Economic Score, Business Ethics, Corporate Governance, Information Security, Cybersecurity & System Availability, Innovation Management, Materiality, Policy Influence, Risk & Crisis Management, Tax Strategy, Transparency & Reporting)
 - The assessment_year is an optional parameter referring to the year when an annual assessment was completed; this parameter is only applicable to the above ESG Analytics data points. Input any year in YYYY format to retrieve the data. If no assessment_year is provided, the latest data is provided.
 - The following data points are retrieved for Climate & Environmental Analytics: tcIRDrtNIndrtCost, tcABSUnpricedCarbonTotal, currentAlignment, compositeScore.

5. `get_stock_price(identifiers*: list, token: str) -> pd.DataFrame:`

- **Purpose:** Retrieves one year (rolling) of historical stock price data including open, close, high, low, and volume.
- **Usage:** Use this function to obtain stock prices and volume for one year.
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: IQ_CLOSEPRICE, IQ_OPENPRICE, IQ_HIGHPRICE, IQ_LOWPRICE, IQ_VOLUME.

6. `get_multiples(identifiers*: list, token: str) -> pd.DataFrame:`

- **Purpose:** Returns key valuation multiples for the latest fiscal year including P/E Ratio, Price/Book, and TEV/EBITDA, to assess a company's market standing.
- **Usage:** Use this function to obtain the latest multiples for the most current fiscal period.
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: IQ_PE_EXCL, IQ_PE_EXCL_FWD_CIQ, IQ_PBV, IQ_PTBV, IQ_TEV_TOTAL_REV, IQ_TEV_EBITDA, IQ_TOTAL_DEBT_EBITDA.

7. `get_estimates(identifiers*: list, token: str) -> pd.DataFrame:`

- **Purpose:** Retrieves analysts' earnings, revenue, and EBITDA estimates across different fiscal periods (IQ_FQ, IQ_FQ+1, IQ_FY, IQ_FY+1, IQ_NTM).
- **Usage:** Use this function to obtain key estimates data alongside recommendations and forward multiples.
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are: IQ_AVG_BROKER_REC_CIQ, IQ_AVG_BROKER_REC_NO_CIQ, IQ_PRICE_TARGET_CIQ, IQ_TARGET_PRICE_NUM_CIQ, IQ_PE_EXCL_FWD_CIQ, IQ_PBV_FWD_CIQ,

IQ_PRICE_CFPS_FWD_CIQ, IQ_PEG_FWD_CIQ, IQ_TEV_EBIT_FWD_CIQ,
IQ_EPS_NORM_EST_CIQ, IQ_REVENUE_EST_CIQ, IQ_EBITDA_EST_CIQ.

- The output response includes:
 1. **Analyst Recommendations & Target Price**
 2. **Forward-Looking Valuation Multiples:** P/E, P/BV, PEG, and TEV/EBIT ratios.
 3. **Forward-Looking Estimates:** Earnings Per Share (EPS), Revenue, and EBITDA

8. `get_agency_ratings(identifiers*: list, token: str) -> pd.DataFrame:`

- **Purpose:** Provides current issuer ratings and outlooks assigned by S&P Ratings, Moody's Investor Services, and Fitch Ratings, providing a one stop-shop for credit ratings.
- **Usage:** Use this function to obtain and compare the ratings from multiple agencies.
- This function does not retrieve data points, but instead retrieves pre-defined data points. More information on these data points can be found on the XpressAPI Swagger page for [Agency Ratings](#).
 - **S&P Ratings:** orgDebtType, ratingType, ratingValue, ratingDate, lastReviewDate, creditWatchOutlookValue, creditWatchOutlookDate.
 - **Moody's Ratings:** ratingTypeText, ratingClassText, ratingValue, ratingDate, lastReviewDate
 - **Fitch Ratings:** ratingTypeName, ratingValue, ratingEffectiveDate, lastReviewDate

9. `get_risk_gauge_score(identifiers*: list, scoring_date: str, token: str) -> pd.DataFrame:`

- **Purpose:** Returns the risk gauge scores for a given date (most recent by default).
- **Usage:** Use this function to obtain the risk gauge score for a given date (most recent by default). Inputs for the `scoring_date` parameter can be found in the [Properties Matrix](#).
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are:
IQ_RG_STANDALONE_CREDIT_SCORE, IQ_RG_STANDALONE_ONE_TO_HUNDRED,
IQ_RG_SOV_CREDIT_SCORE, IQ_RG_SOV_ONE_TO_HUNDRED,
IQ_RG_PG_CREDIT_SCORE, IQ_RG_PG_ONE_TO_HUNDRED.

10. `get_transactions(identifiers*: list, token: str) -> pd.DataFrame:`

- **Purpose:** Returns data on the top 5 latest transactions based on the requested identifiers.
- **Usage:** Use this function to obtain the top 5 latest transactions and associated deal information. The list of transactions are returned chronologically starting from the most recent transaction.
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are:
IQ_TRANSACTION_LIST, IQ_TR_TRANSACTION_TYPE, IQ_TR_DEAL_RESOLUTION,
IQ_TR_TOTALVALUE, IQ_TR_ANN_DATE, IQ_TR_CLOSED_DATE.
- The function output includes:
 1. **Transaction List:** A ranked list of recent transactions.
 2. **Transaction Type:** Classification such as M&As, Buybacks, Public Offerings, or Private Placements.

3. **Deal Resolution:** Summary of the transaction details.
4. **Transaction Value:** Gross value of the transaction.
5. **Announced & Completed Dates:** Key dates marking the transaction timeline.

```
11. get_latest_activity(identifiers*: list, token: str) -> pd.DataFrame:
```

- **Purpose:** Provides a comprehensive overview of the most recent corporate activities, including recent documents, past and upcoming events, and key developments.
- The function output includes:
 1. **Documents (Top 10 Most Recent):** A ranked list of recently published reports, filings, and transcripts for the company.
 2. **Events (Top 3 Historical & Top 3 Future):** A timeline of major corporate events, such as earnings calls, investor conferences, and financial disclosures.
 3. **Key Developments (Top 5 Most Recent):** A summary of the most important announcements, collaborations, acquisitions, and business updates etc.
- **Usage:** Use this function to retrieve information on recent corporate activities.
- **Mnemonics:** This function retrieves a pre-defined set of mnemonics; more information on these mnemonics can be found within the [API Mnemonics List](#). These mnemonics are:
- The function output includes:
 1. **Documents:** Documents and Transcripts with the respective date. The data items are: filingDate, keyDevDate, fileType. More information on these data points can be found on the Swagger page for Physical Documents.
 2. **Events:** IQ_EVENT_ID, IQ_EVENT_DATE, IQ_EVENT_TYPE
 3. **Key Developments:** IQ_KEY_DEV_ID, IQ_KEY_DEV_DATE, IQ_KEY_DEV_SITUATION, IQ_KEY_DEV_TYPE

Company Name Match Functions

Overview

This section provides information on the Kensho Link Function which is used to map a company name to its entity-level identifier and the associated tickers. Functions will return data in the form of pandas DataFrames.

Functions

```
1. get_company_name_to_id (companies*: list, identifier_type: str, token: str) -> pd.DataFrame:
```

- **Purpose:** Returns a list of closely matched for a user input company name. Information returned includes companyId / institutionId, company names, and tickers of the five closest matches. Input "ciq" or "mi" for identifier_type. If identifier_type is not provided, the ciq companyId is retrieved.
- **Usage:** Use this function to map a company name to its entity-level identifier and ticker.

This page is intentionally left blank.

Using the SDK & Code Samples

Set Up

1. Importing Required Classes

To use the SDKs, you need to import certain classes. These classes are necessary in order to use the various functions within the SDK from authentication to the retrieval of market and financial data.

The below code demonstrates how to import the required classes:

```
from spgmi_api_sdk.ciq.services import SDKDataServices
```

2. Using a Proxy (Optional)

An optional step includes using a proxy. A proxy enables internet access through an alternate server. This can be necessary if your organization uses a proxy for security or network management.

The below code demonstrates the structure required to set up a proxy with a username, password, and server address. If you're in a typical network environment without special requirements, you don't need to set up a proxy.

```
from spgmi_api_sdk.ciq.services import SDKProxy # necessary import

sdk_proxy_object = SDKProxy(proxy_username="", proxy_password="",
                             proxy_host=None, proxy_port=None, proxy_domain="") # object creation
```

3. Required Instances

The below code demonstrates how to set up required instances. Instances are objects you need to use the SDK for different purposes including using specific functions and services for authentication, financial data, market data, data widget functions, and the company name match function.

```
spg = SDKDataServices(username="your_api_username",
    password="your_api_password", proxy=sdk_proxy)
```

4. Extracting the Token Independently (Optional)

If you want to extract the token independently and pass it to individual methods, you can use the below code. This is optional, and the previous step will authenticate you.

Use the 'get_token' method to retrieve a bearer token.

```
token = spg.get_token().get('access_token')
```

Fetching Data

Financials Code Samples

Below, we have provided code samples for each Financials function within this library. For more information on the purpose of each function and supported properties, see the [Financial Services Functions](#) section.

1. "get_income_statement_pit"

```
response =
spg.get_income_statement_pit(identifiers=["I_US5949181045", "2588173", "EG13
20"], properties={"asOfDate": "12/31/2020", "currencyId":
"USD", "currencyConversionModeId": "HISTORICAL"})
```

2. "get_income_statement_historical"

```
response =
spg.get_income_statement_historical(identifiers=["GV012141", "MSFT:NasdaqGS
"], properties={"periodType": "IQ_FQ-4"})
```

3. "get_balance_sheet_pit"

```
response = spg.get_balance_sheet_pit(identifiers=["RX309198", "MMM: "],
properties={"asOfDate": "12/31/2020", "currencyId":
"USD", "currencyConversionModeId": "HISTORICAL"})
```

4. "get_balance_sheet_historical"

```
response =
spg.get_balance_sheet_historical(identifiers=["I_US5949181045", "2588173"],
properties={"periodType": "IQ_FQ-2"})
```

5. "get_cash_flow_pit"

```
response = spg.get_cash_flow_pit(identifiers=["2588173","EG1320"],
properties={"asOfDate": "12/31/2020", "currencyId":
"USD", "currencyConversionModeId": "HISTORICAL"})
```

6. "get_cash_flow_historical"

```
response =
spg.get_cash_flow_historical(identifiers=["MSFT:NasdaqGS", "DB649496569"],
properties={"asOfDate": "12/31/2020", "currencyId":
"USD", "currencyConversionModeId": "HISTORICAL"})
```

7. "get_financials_pit"

```
response =
spg.get_financials_pit(identifiers=["I_US5949181045", "2588173", "EG1320", "C
SP_594918104", "IQT2630413", "GV012141", "MSFT:NasdaqGS", "DB649496569", "RX309
198"], mnemonics=["IQ_CASH_INVEST_NAME_AP"], properties={"asOfDate":
"12/31/2020", "currencyId": "USD", "currencyConversionModeId":
"HISTORICAL"})
```

8. "get_financials_historical"

```
response =
spg.get_financials_historical(identifiers=["I_US5949181045", "2588173", "EG1
320", "CSP_594918104", "IQT2630413", "GV012141", "MSFT:NasdaqGS", "DB649496569"
, "RX309198"], mnemonics=["IQ_CASH_INVEST_NAME_AP"],
properties={"asOfDate": "12/31/2020", "currencyId":
"USD", "currencyConversionModeId": "HISTORICAL"})
```

Market Data Code Samples

Below, we have provided code samples for each Market Data function within this library. For more information on the purpose of each function and supported properties, see the Market Data Functions section.

1. "get_pricing_info_pit"

```
response =
spg.get_pricing_info_pit(identifiers=["I_US5949181045", "2588173", "EG1320",
"CSP_594918104", "IQT2630413"], properties={})
```

2. "get_pricing_info_time_series"

```
response =
spg.get_pricing_info_time_series(identifiers=["CSP_594918104", "IQT2630413"
, "GV012141", "MSFT:NasdaqGS", "DB649496569", "RX309198", "MMM: "],
properties={})
```

3. "get_dividend_info_pit"

```
response
=spg.get_dividend_info_pit(identifiers=["CSP_594918104","IQT2630413","GV01
2141","MSFT:NasdaqGS"], properties={})
```

4. "get_dividend_info_time_series"

```
response =
spg.get_dividend_info_time_series(identifiers=["GV012141","MSFT:NasdaqGS",
"DB649496569","RX309198","MMM:"], properties={})
```

5. "get_market_info_pit"

```
response =
spg.get_market_info_pit(identifiers=["IQT2630413","GV012141","MSFT:NasdaqG
S","DB649496569","RX309198"], properties={})
```

6. "get_market_info_time_series"

```
response = spg.get_market_info_time_series(identifiers=["AAPL:"],
properties={})
```

By using these methods and the `IPython.display.display` function, you can efficiently retrieve and display various types of market data across a specific point in time or a range of dates.

CIQ Data Widget & Company Name Match Code Samples

Below, we have provided code samples for each function associated with the CIQ Data Widgets and other reference data. For more information on the purpose of each function and supported properties.

1. "get_financials"

```
response =
spg.get_financials(identifiers=["IQ21835","IQ26642","MSFT"],
period_type="IQ_FY-1")
```

2. "get_market_data"

```
result = spg.get_market_data(identifiers=["IQ112350","MSFT","IQ24937"])
```

3. "get_corporate_data"

```
response =
spg.get_corporate_data(identifiers=["IQ112350","SPGI","IQ24937"])
```

4. "get_esg_scores"

```
response = spg.get_esg_scores(identifiers=["SNL4202062"],assessment_year=
"2022")
```

5. "get_company_name_to_id"

```
response = spg.get_company_name_to_id(companies=["Coca Cola", "Meezan Bank Limited (MEBL)"], identifier_type="ciq")
```

6. "get_stock_price"

```
response = spg.get_stock_price(identifiers=["IQ112350", "MSFT", "IQ24937"])
```

7. "get_multiples"

```
response = spg.get_multiples(identifiers=["SNL4004214", "MSFT"])
```

8. "get_estimates"

```
response = spg.get_estimates(identifiers=["IQ112350", "MSFT", "IQ24937", "NVDA", "SNL4094286", "SNL"])
```

9. "get_agency_ratings"

```
response=spg.get_agency_ratings(identifiers=["IQ112350", "IQ24937"])
```

10. "get_risk_gauge_score"

```
response = spg.get_risk_gauge_score(identifiers=["IQ112350", "MSFT", "IQ24937", "NVDA", "SNL4094286", "SNL"], scoring_date="06/03/2025")
```

11. "get_transactions"

```
response =  
spg.get_transactions(identifiers=["IQ112350", "IQ162270", "MSFT", "sadfa", "IQ32307", "IQ32307", "NVDA", "SNL4094286"])
```

12. "get_latest_activity"

```
response_list=spg.get_latest_activity(identifiers=["IQ112350", "IQ24937"])  
documents,events,key_devs = response_list
```

Complete Example Code

Below is an example of a full request which includes all steps outlined in previous sections including importing the necessary classes, using a proxy (optional), setting up required instances, completing authentication, and retrieving both financial and market data with the appropriate functions and displaying data in DataBricks, Jupyter Notebooks, PyCharm, and IDE.

The comments indicate each step:

```
#necessary import  
from spgmi_api_sdk.ciq.services import SDKDataServices  
#optional import  
from spgmi_api_sdk.ciq.services import SDKProxy  
#instatiation
```

```

sdk_proxy_object = SDKProxy(proxy_username="", proxy_password="",
proxy_host=None, proxy_port=None, proxy_domain="") #optional

spg = SDKDataServices(username="your_api_username",
password="your_api_password", proxy=sdk_proxy_object) #mandatory

#for manually fetching token (optional)
token=spg.get_token().get('access_token')

#Fetching data
response =
spg.get_financials(identifiers=["IQ21835","IQ26642","MSFT"],
period_type="IQ_FY-1")
print(response)
response =
spg.get_market_data(identifiers=["IQ112350","MSFT","IQ24937"])
print(response)
response =
spg.get_corporate_data(identifiers=["IQ112350","SPGI","IQ24937"])
print(response)
response =
spg.get_esg_scores(identifiers=["SNL4202062"],assessment_year= "2022")
print(response)
response = spg.get_multiples(identifiers=["SNL4004214","MSFT"])
print(response)
response = spg.get_risk_gauge_score(identifiers=["IQ112350", "MSFT",
"IQ24937", "NVDA", "SNL4094286", "SNL"], scoring_date="06/03/2025")
print(response)
response = spg.get_estimates(identifiers=["IQ112350", "MSFT",
"IQ24937", "NVDA", "SNL4094286", "SNL"])
print(response)
response =
spg.get_transactions(identifiers=["IQ112350","IQ162270","MSFT","sadfa",
"IQ32307","IQ32307","NVDA","SNL4094286"])
print(response)
response = spg.get_stock_price(identifiers=["IQ112350", "MSFT",
"IQ24937"])
print(response)
response = spg.get_agency_ratings(identifiers=["IQ112350", "IQ24937"])
print(response)
response_list=spg.get_latest_activity(identifiers=["IQ112350",
"IQ24937"])
documents,events,key_devs = response_list
print(documents)
print(events)
print(key_devs)
response = spg.get_company_name_to_id(companies=["Coca Cola", "Meezan
Bank Limited (MEBL)"])
print(response)

```

```
#Fetching Data with manually sent token
response =
spg.get_financials(identifiers=["IQ21835","IQ26642","MSFT"],
period_type="IQ_FY-1", token= token)
print(response)
```

Verifying SDK File Integrity

To ensure the integrity of the downloaded SDK file, follow the outlined steps to verify its authenticity. This method demonstrates how to verify the hash value of the file using various tools on different platforms.

Compute the SHA-256 Hash

- Windows: Use the Command Prompt and the `CertUtil` command:
 - Execute the command: `certutil -hashfile <path_to_tar.gz> SHA256`
- Unix-like Systems (Linux, macOS): Use either Terminal with `shasum` or `sha256sum`
 - With `shasum`, execute : `shasum -a 256 <path_to_tar.gz>`
 - With `sha256sum`, execute : `sha256sum <path_to_tar.gz>`

Replace “<path_to_tar.gz>” with the full path to SDK file. This will compute the SHA-256 hash of file.

Verify the Hash

- Open the `checksum.sha256` file from downloaded zip file to check the expected SHA-256 hash.
- Ensure that the computed hash matches the value in `checksum.sha256` file.

If the hashes match, the file is intact and authentic.

If the hashes do not match, the file may have been corrupted or tampered with. If this happens, please re-download the file or contact Enterprise API Support team for assistance.

By following these steps, you can confidently verify that the SDK file is genuine and has not been altered.

Best Practices

Use of Environment Variables and .env Files

To maintain security and adhere to best practices when working with sensitive information such as usernames and passwords in Jupyter Notebooks, consider the following guidelines:

Use of .env Files

Create a .env File: Store your sensitive information, such as username and passwords, in an .env file located in the root directory of your project.

```
SDK_USERNAME="--username--"
SDK_PASSWORD="--password--"
```

Load .env Variables

Utilize the python-dotenv library to load these variables into your notebook session securely.

```
# Best Practice: Secure Credential Management in Jupyter Notebooks

# Store and Retrieve Credentials from a .env File

import os
from dotenv import load_dotenv

# Load environment variables from .env file, assume file name is 'py.env'
load_dotenv('py.env')

# Access the variables
username = os.getenv('SDK_USERNAME')
password = os.getenv('SDK_PASSWORD')

# Example usage
token_response = auth_services.get_token(username, password)
```

Preferred Identifier Types

Although the SDK supports a range of identifier types, using CIQ identifiers results in improved response time for most functions. As an exception, the ESG Scores function has a better response time when MI identifiers are used. The `get_company_name_to_id` function enables you to obtain both CIQ and MI identifiers for any searched company.

Throttling Limits

In cases when you have reached your throttling limit, you will receive a 429 error code: “____ Request Limit of ____ Exceeded” showing the frequency (daily, monthly) and the number of requests allowed.

Additional Notes

- **Maximum Identifiers:** All functions accept a maximum of 10 identifiers. For the ESG Scores function, only one identifier is supported.
- **Maximum Mnemonics:** All financial data fetching functions with mnemonic as an input parameter accept a maximum of 10 mnemonics. Refer to the [API Mnemonics List](#) and filter by `S&P_Capital_IQ_Financials` in the fulfilment column for a full list of supported mnemonics.
- **Properties:** A list of supported properties can be found beneath each function in the Overview of Functions section. For information on specific properties, refer to the [Properties Matrix](#) document.
- **Response time:** Some functions may internally invoke multiple API calls, causing a slight response time delay.

Revision History

The changes made to this document include the following:

Version	Date	Changes
v1.1	03/28/2025	Added new content for new version of Python SDK including new market and financial data functions and new data widget functions.
v1.0	07/31/2024	Initial version